



Nombre: Luis Alberto Vargas González.

Fecha: 07/07/2025

Actividad y unidad: U8 A1 Resumen de la teoría de la complejidad

Clase: Diseño y Análisis de Algoritmos.

Maestría en Ciberseguridad.

Introducción.

En esta actividad se desarrolló un resumen de la teoría de la complejidad , en donde pudimos observar temas que en un principio son complejos y no fáciles de asimilar , pero que después de bastantes horas de estudio y práctica, podemos empezar a entender de mejor manera ya que si bien esta teoría fundamenta las ciencias computacionales y es base importante de los grandes problemas de optimización, de cálculos infinitamente complejos , también es base de los compiladores y de los intérpretes que son una tecnología fundamental para poder usar lenguajes de alto nivel como Python, Java, C++, y cualquier otro lenguaje, incluidos los de instrucciones como Bash, scripts y demás.

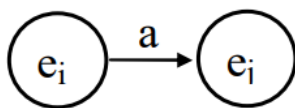
Personalmente mi tema favorito dentro de esta categoría son los autómatas finitos , tanto deterministas como no-deterministas , pues personalmente pude estudiar durante varias semanas durante la universidad estos temas a profundidad y usar programas de construcción y simulación de autómatas como JFlap escrito en Java usando bibliotecas gráficas, en donde podíamos probar distintas cadenas y alfabetos para probar el diseño de los mismos, y de ser admitidas estas cadenas, significaba que dicho diseño previamente construido , era correcto.

Desarrollo.

Autómata finito.

Es un modelo matemático de una máquina que acepta cadenas de un lenguaje definido sobre un Alfabeto A. Consiste en un conjunto de estados y de transiciones entre estos estados , normalmente se les llama “q” a los estados , de los cuales dependen los símbolos de entrada de la cadena. Una cadena puede ser por ejemplo un código binario como “11101101100”.El autómata aceptara esta cadena si el último de los dígitos que tenemos (en este caso el ultimo 0), nos lleva a un estados final. Se dice que un autómata es determinista si para cada estado hay existe máximo 1 sola transición. Siendo los no-deterministas los que tendrán 2 transiciones o más entre estados.

Normalmente existe un “gráfico” asociado a cada autómata , este grafico es un grafo dirigido, el cual vimos anteriormente. El estado inicial se indica mediante una flecha que no tiene nodo origen. Los estados finales se representan con un círculo doble. Si existe una transición del estado e_i al estado e_j para un símbolo de entrada a, existe entonces un arco rotulado a desde el nodo e_i al nodo e_j ; es decir que $\delta(e_i, a) = e_j$, se representa en el diagrama

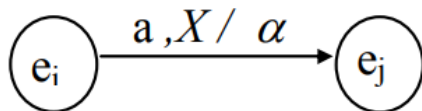


Autómata de Pila.

Asi como los autómatas finitos aceptan cadenas de un cierto alfabeto , asi también pueden aceptar los de pila, pero los de pila son un poco mas poderoso que los finitos ya que puede aceptar lenguajes que los finitos no pueden. Uno de pila tiene una estructura de datos tipo cinta de entrada y un mecanismo de control que puede encontrarse en uni

de entre un numero finito de estados. Uno de estos estados es el designado como inicial y los demás son designados como de aceptación o finales. A diferencia de los autómatas finitos, los autómatas de pila cuentan con una memoria auxiliar llamada pila. Los símbolos (llamados símbolos de pila) pueden ser insertados o extraídos de la pila, de acuerdo con el manejo last-in-first-out (LIFO). Las transiciones entre los estados que ejecutan los autómatas de pila dependen de los símbolos de entrada y de los símbolos de la pila. El autómata acepta una cadena x si la secuencia de transiciones, comenzando en estado inicial y con pila vacía, conduce a un estado final, después de leer toda la cadena x .

La función de transición de estados de un AP puede ser representada por un diagrama donde los nodos representan los estados y los arcos transiciones. Si existe transición tipo (1), el arco queda rotulado de la siguiente manera:

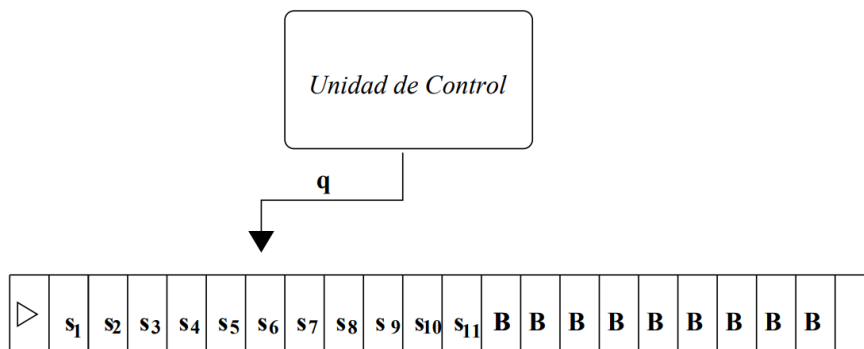


Si el estado actual es e_i y la cabeza lectora apunta un símbolo a , y el tope de la pila es X , entonces cambiar al nuevo estado e_j , avanzar la cabeza lectora, y sustituir el símbolo del tope X en la pila por la cadena α . Por ejemplo: Si $\alpha = ZYX$ deja X , apila Y , y apila Z (nuevo tope Z). donde $X, Y, Z \in P$ Si $\alpha = XX$ deja X y apila X (nuevo tope X). Si $\alpha = X$ deja X como el mismo tope (no altera la pila) Si $\alpha = \epsilon$ elimina X , y el nuevo tope es el símbolo por debajo (desapila).

Máquina de Turing determinista.

De manera informal una máquina de Turing esta compuesta de :

1. Una cinta infinita estructurada en casillas enumeradas (Con primera casilla). Cada una conteniendo un símbolo del alfabeto que se trabajará.
2. Un cabezal de trabajo que actuara como lector y escritor que inspecciona el contenido de una casilla (lee). Puede cambiar su contenido por otro (escribe) y además se desplaza a lo largo de la cinta (una casilla a la derecha o una a la izquierda o se queda dónde está)



Máquina de Turing no determinista.

Para entender correctamente esta maquina debemos de anotar lo que es el determinismo y el no determinismo; En los sistemas deterministas tenemos la información precisa del estado del sistema, el cual nos permite predecir el futuro del mismo, con precisión casi infinita. Estos sistemas son casi siempre simplificaciones del mundo real o sistemas creados y diseñados por el ser humano.

En cambio, en el no determinismo, no tenemos la información suficiente para predecir estados futuros del sistema de manera precisa; algunos ejemplos de estos son; el clima, comportamiento de una persona, comportamiento de mercado financiero.

El comportamiento de una máquina de Turing **no determinista** es hasta cierto grado impredecible. Incluso con las mismas entradas y estados, la máquina puede devolver diferentes resultados en diferentes ejecuciones.

A diferencia de los sistemas no deterministas de los que hablamos arriba, las máquinas de Turing no deterministas no es que tengan una probabilidad de pasar de un estado a otro, sino que la máquina puede “seleccionar” una transición de un estado a otro.

Clases de complejidad.

Clase de complejidad P: Llamada también la de los problemas de decisión, en donde podemos dar respuestas de Si o No, estos son los problemas algorítmicos de clase polinomial, Con esto nos referimos a que debe de existir un entero $k \geq 0$ y un algoritmo tal que cualquier instancia de tamaño n la resuelva en $O(n^k)$.

Clase de complejidad NP: Son aquellos problemas cuya respuesta es “sí” se puede verificar en tiempo polinomial. O dicho de otro modo, cada vez que haya un “sí” debe de haber un **objeto testigo** que testifique la respuesta y un **algoritmo verificador**. Que cheque la correctitud del testigo en tiempo polinomial.

El problema P vs NP.

Será $P = NP$?

Esa es la gran pregunta, ya que como se pudo constatar anteriormente, todo lo que existe en NP puede y debe existir en P, pero no sabemos a ciencia cierta si lo es por igual al revés, es decir si todo lo de NP esta en P; Muchos de los problemas NP-completos son fundamentales en varias áreas. Miles de personas, incluidos los expertos de dichas áreas, se los han encontrado y han intentado resolverlos en tiempo polinomial. Hasta ahora, nadie ha logrado resolver ninguno de ellos en tiempo polinomial. Ni nadie ha demostrado que para alguno de ellos esto sea imposible. Como son problemas NP-completos, basta con que uno de ellos esté en P para que todos lo estén. Si esto llegara a suceder, las clases de complejidad P y NP serían la misma y por lo tanto en un sentido muy amplio sería igual responder problemas que verificar su solución.

Conclusión.

Podemos argumentar que ; a manera de resumen , se cumplió satisfactoriamente con este requerimiento, cubriendo la totalidad de los temas ya vistos en este trabajo, si bien como resumen , no se pueden cubrir todos los aspectos profundos, detalles técnicos y aspectos formales de cada tema, ya que eso nos impediría generar un buen resumen , y estaríamos generando un documento técnico completo, lo cual puede resultar contraproducente.

Además es válido recalcar que para una investigación más profunda de los temas , es necesario poder contar con libros especializados y cursos especializados como los que nos fueron presentados tanto en videos , asi como en las secciones que fueron consultadas para referencias bibliográficas de este mismo documento.

Referencias Bibliográficas.

1. *Apuntes de Computación*. Universidad del Centro.; 2009.
<https://users.exa.unicen.edu.ar/catedras/ccomp1/ApunteAutomatasFinitos.pdf>.
Accedido julio 7, 2025.
2. *Apuntes de Autómatas de pila*. Universidad del Centro; 2008.
<https://users.exa.unicen.edu.ar/catedras/ccomp1/Apunte4.pdf>. Accedido julio 7, 2025.
3. Jiménez Pérez M de J. Modelos de computación; Máquinas de Turing.
<https://www.cs.us.es/~marper/docencia/TCC-2019-2020/temas/tema-2-trans.pdf>.
Published 2019.
4. Héctor Patricio. Máquinas de Turing no deterministas y problemas NP. *The Dojo MX Blog*. febrero 2023. <https://blog.thedojo.mx/2023/02/08/maquinas-de-turing-no-deterministas-y-problemas-np.html>.
5. Clases de complejidad y P vs. NP — Matemáticas Discretas para Ciencia de Datos. <https://madi.nekomath.com/P5/ReduccionesPvsNP.html>.